

layout. Dropping down to a singular column will give the content of each panel the space it needs to render without looking cramped on the page of smaller viewports.

When a grid has no panels it should render an empty state.

Our preferred choice for this grid is to use Gridstack, an open-source MIT licensed library which supports grid structures, along with the changing and resizing of grid items, in a deterministic, and cross-browser friendly way.

The framework configuration must not tie directly to Gridstack in case we need to move to alternatives in future.

Any grid configuration options to be abstracted to make it easier to migrate if required.

Panels

Panels are the wrapping modular container that provides a contextual interface for users to interact with their data.

Each panel contains:

- A title
- A tooltip for further tertiary information (optional)
- A kebab menu of contextual actions (optional)
- An indicator for any contextual errors/warnings/info (optional)
- A loading state whilst the panel retrieves the visualization data (optional)
- The content area

Each panel can be resized in accordance with the grid and are scrollable when the content exceeds the size of the panel.

Visualizations

Visualizations refer to any component that renders a graphical representation of the data. The dashboard layout framework is agnostic to the content of the visualization, and how the visualization retrieves and processes the data.

Some common examples include:

- ECharts-based visualizations, using the GitLab UI implementations
 - Visualizations may contain axis, legends, and other clickable elements depending upon the ECharts implementation.
- Tables, using GitLab UI (or the lite version)
 - Visualizations may contain keyset pagination, sorting, and internal searching.
- Text or markdown-based content.

Filters

Although filtering as a concept is still under heavy UX exploration, we can be assured that there will be two slots where filters can be added to the dashboard layout, and used by

the user.

The first slot is the global filter that is applied to every visualization within the dashboard. This will allow engineers to define the filters that will apply to all the panels on the dashboard. It is up to the engineer to make sure that filters are applied appropriately to their visualizations.

In the event that a filter does not work with a visualization, then the user needs to be notified through the UI. How this notification looks is still being discussed.

..

The second type of filter is a per-panel filter applied to any one individual panel. At this moment, the dashboard layout framework only supports global filters. Per-panel filters will also need to be connected to the visualization by the engineer.

Applying a filter will automatically send all applied filters to every applicable panel and visualization. For instances where filters need to be synced to the URL, engineers can make use of the UrlSync component.

For engineers developing panels, there is also scope to pre-apply filters to a panel, and then either allow these to be changed by the user or for them to be restricted to only those values. If a panel is "locked" to a particular filter value, for instance, a panel which can only show a visualization of data over the past 24 hours, the panel will visually indicate to the user that it is "locked" and explain what is restricted.

Error handling

In the event of an error, the UI should display a useful error message and include corrective steps where available.

When possible, errors should provide a retry button, or another next-step for users.

Page-level errors should use the Pajama's alert component.

The errors should be located where the error occurred:

- Entire dashboard - replace dashboard grid with the error message.
- Global filters - show above the global filters unless the filter selection causes a specific error for a panel, in which case the error will appear within the panel error state.

For per-panel filters and panel-specific errors, we should use the panel error state. The panel error state will:

- Highlight the panel has an error through color and iconography.
- Have a tooltip explaining the error in detail, and what the user can do, with a retry button where applicable.
- Replace the panel content with a contextual error message.

Getting started

See the development documentation.

Migrating existing dashboards

See the development documentation.

SECTION: engineering/architecture/design-documents/database_size_limits/_index.md

<!-- vale gitlab.FutureTense = NO -->

{{< engineering/design-document-header >}}

This document is a proposal to work towards reducing and limiting table sizes on GitLab.com. We establish a measurable target by limiting table size to a certain threshold (100 GB). Action however should be taken as early as 10 GB.

Size limits are used as an indicator to drive database focus and decision making.

With GitLab.com growing, we continuously re-evaluate which tables need to be worked on to prevent or otherwise fix violations.

This is not meant to be a hard rule but rather a strong indication that work needs to be done to break a table apart or otherwise reduce its size.

This is meant to be read in context with the Database Sharding blueprint, which paints the bigger picture. This proposal here is thought to be part of the "debloating step" below, as we aim to reduce storage requirements and improve data modeling. Partitioning is part of the standard tool-belt: where possible, we can already use partitioning as a solution to cut physical table sizes significantly. Both help to prepare efforts like decomposition (database usage is already optimized) and sharding (database is already partitioned along an identified data access dimension).

mermaid

graph LR

Fe(Pick feature) --> D

D[(Database)] --> De

De[Debloating] --> Dc

Dc[Decomposition] --> P

P[Partitioning] --> S

S[Sharding] --> R

P --> M

M[Microservices] --> R

R{Repeat?} --> Fe

style De fill:fca326

style P fill:fc6d26

Motivation: GitLab.com stability and performance

Large tables on GitLab.com are a major problem - for both operations and development. They cause a variety of problems:

1. Query timings and hence overall application performance suffers

1. Table maintenance becomes much more costly. Vacuum activity has become a significant concern on GitLab.com - with large tables only seeing infrequent (once per day) processing and vacuum runs taking many hours to complete. This has various negative consequences and a very large table has potential to impact seemingly unrelated parts of the database and hence overall application performance suffers.

1. Data migrations on large tables are significantly more complex to implement and incur development overhead. They have potential to cause stability problems on GitLab.com and take a long time to execute on large datasets.

1. Indexes size is significant. This directly impacts performance as smaller parts of the index are kept in memory and also makes the indexes harder to maintain (think repacking).

1. Index creation times go up significantly - in 2021, we see B-Tree creation take up to 6 hours for a single B-Tree index. This impacts our ability to deploy frequently and leads to vacuum-related problems (delayed cleanup).

1. We tend to add many indexes to mitigate, but this eventually causes significant overhead, can confuse the query planner and a large number of indexes is a smell of a design problem.

Target: All physical tables on GitLab.com are < 100 GB including indexes

To maintain and improve operational stability and lessen development burden, we target a table size less than 100 GB for a physical table on GitLab.com (including its indexes). This has numerous benefits:

1. Improved query performance and more stable query plans

1. Significantly reduce vacuum run times and increase frequency of vacuum runs to maintain a healthy state - reducing overhead on the database primary

1. Index creation times are significantly faster (significantly less data to read per index)

1. Indexes are smaller, can be maintained more efficiently and fit better into memory

1. Data migrations are easier to reason about, take less time to implement and execute

See also <<https://postgres.fm/episodes/partitioning/transcript>> (search for hundred in the transcript).

It is much easier to rectify problems when the table is small. Do not wait until the table approaches 100 GB.

Rather, start action to reduce table sizes when the table is around 10 GB.

This target is pragmatic: We understand table sizes depend on feature usage, code changes and other factors - which all change over time. We may not always find solutions where we can tightly limit the size of physical tables once and for all. That is acceptable though and we primarily aim to keep the situation on GitLab.com under control. We adapt our efforts to the situation present on GitLab.com and re-evaluate frequently.

While there are changes we can make that lead to a constant maximum physical table size over time, this doesn't need to be the case necessarily. Consider for example hash partitioning, which breaks a table down into a static number of partitions. With data growth over time, individual partitions also grow in size and may eventually reach the threshold size again. We strive to get constant table sizes, but it is acceptable to ship easier solutions that don't have this characteristic but improve the situation for a considerable amount of time.

As such, the target size of a physical table after refactoring depends on the situation and there is no hard rule for it. We suggest to consider historic data growth and forecast when physical tables reach the threshold of 100 GB again. This allows us to understand how long a particular solution is expected to last until the model has to be revisited.

NOTE:

In PostgreSQL context, a physical table is either a regular table or a partition of a partitioned table.

Examples

Most prominently, the cibuilds table is 1.5 TB in size as of June 2021 and has 31 indexes associated with it which sum up to 1 TB in size. The overall on-disk size for this table is 2.5 TB. Currently, this grows at 300 GB per month. By the end of the year, this is thought to be close to 5 TB if we don't take measures against.

The following examples show that very large tables often constitute the root cause of incidents on GitLab.com.

1. Infrequent and long running vacuum activity has led to repeated degradation of query performance for CI queuing
1. On large tables like cibuilds, index creation time varies between 1.5 to 6 hours during busy times. This process blocks deployments as migrations are being run synchronously - reducing our ability to deploy frequently.
1. Creating a large index can lead to a burst of activity on the database primary:
 1. on mergerequestdiffcommits table: caused high network saturation,
 1. regular reindexing activity on the weekend: causes growing WAL queue (impacts recovery objectives),
 1. notes table: Re-creating a GIN trigram index for maintenance reasons has become nearly unfeasible and had to be aborted after 12 hours upon first try as it was blocking other vacuum operation.

Problematic tables on GitLab.com

NOTE: See <https://gitlab-com.gitlab.io/gl-infra/platform/stage-groups-index/> for up to date table sizes.

This shows the TOP30 tables by their total size (includes index sizes) as of mid June 2021 on GitLab.com. tablesize, indexsize is the on-disk size of the actual data and associated indexes, respectively. percentageoftotaldatabasesize displays the ratio of total table size to database size.

As we can see, there are currently very large tables greater than 1 TB in size, which also tend to have very large indexes.

The other observation here is that there are also tables with a large number of indexes and total index size can be significantly larger than the data stored. For example, deployments is 30 GB in size plus additional 123 GB of index data spread across 24 indexes.

<!--

```

select tablename,
       pgsizepretty(pgtotalrelationsize(t.schemaname || '.' || t.tablename)) as totalsize,
       pgsizepretty(pgrelationsize(t.schemaname || '.' || t.tablename)) as tablesizes,
       pgsizepretty(pgindexessize(t.schemaname || '.' || t.tablename)) as indexsize,
       count() as indexcount,
       round(pgtotalrelationsize(t.schemaname || '.' || t.tablename) /
pgdatabasesize('gitlabhqproduction')::numeric 100, 1) as percentageoftotaldatabasesize
from pgindexes i
join pgtables t USING (tablename)
group by 1,
         2,
         3,
         t.schemaname,
         t.tablename
order by pgtotalrelationsize(t.schemaname || '.' || t.tablename) desc
limit 30;
-->

```

| Table | Total size | Table size | Index size | Index count | Percentage of total database size |
|-------------------------|------------|------------|------------|-------------|-----------------------------------|
| cibuilds | 2975 GB | 1551 GB | 941 GB | 30 | 22.7 |
| mergerequestdiffcommits | 1890 GB | 1454 GB | 414 GB | 2 | 14.4 |
| cibuildtracesections | 1123 GB | 542 GB | 581 GB | 3 | 8.6 |
| notes | 748 GB | 390 GB | 332 GB | 13 | 5.7 |
| mergerequestdifffiles | 575 GB | 481 GB | 88 GB | 1 | 4.4 |
| events | 441 GB | 95 GB | 346 GB | 12 | 3.4 |
| cijobartifacts | 397 GB | 187 GB | 210 GB | 10 | 3.0 |
| cipipelines | 266 GB | 66 GB | 200 GB | 23 | 2.0 |
| taggings | 238 GB | 60 GB | 179 GB | 5 | 1.8 |
| cibuildsmetadata | 237 GB | 88 GB | 149 GB | 5 | 1.8 |
| issues | 219 GB | 47 GB | 150 GB | 28 | 1.7 |
| webhooklogs202103 | 186 GB | 122 GB | 8416 MB | 3 | 1.4 |
| cistages | 182 GB | 58 GB | 124 GB | 6 | 1.4 |
| webhooklogs202105 | 180 GB | 115 GB | 7868 MB | 3 | 1.4 |

| | | | | | |
|---------------------|--------|--------|---------|----|-----|
| mergerequests | 176 GB | 44 GB | 125 GB | 36 | 1.3 |
| webhooklogs202104 | 176 GB | 115 GB | 7472 MB | 3 | 1.3 |
| webhooklogs202101 | 169 GB | 112 GB | 7231 MB | 3 | 1.3 |
| webhooklogs202102 | 167 GB | 111 GB | 7106 MB | 3 | 1.3 |
| sentnotifications | 166 GB | 88 GB | 79 GB | 3 | 1.3 |
| webhooklogs202011 | 163 GB | 113 GB | 7125 MB | 3 | 1.2 |
| pusheventpayloads | 162 GB | 114 GB | 48 GB | 1 | 1.2 |
| webhooklogs202012 | 159 GB | 106 GB | 6771 MB | 3 | 1.2 |
| webhooklogs202106 | 156 GB | 101 GB | 6752 MB | 3 | 1.2 |
| deployments | 155 GB | 30 GB | 125 GB | 24 | 1.2 |
| webhooklogs202010 | 136 GB | 98 GB | 6116 MB | 3 | 1.0 |
| webhooklogs202009 | 114 GB | 82 GB | 5168 MB | 3 | 0.9 |
| securityfindings | 109 GB | 21 GB | 88 GB | 8 | 0.8 |
| webhooklogs202008 | 92 GB | 66 GB | 3983 MB | 3 | 0.7 |
| resourcelabelevents | 66 GB | 47 GB | 19 GB | 6 | 0.5 |
| mergerequestdiffs | 63 GB | 39 GB | 22 GB | 5 | 0.5 |
| | | | | | |

Solutions

There is no standard solution to reduce table sizes - there are many!

1. Retention: Delete unnecessary data, for example expire old and unneeded records.
1. Remove STI: We still use single-table inheritance in a few places, which is considered an anti-pattern. Redesigning this, we can split data into multiple tables.
1. Index optimization: Drop unnecessary indexes and consolidate overlapping indexes if possible.
1. Optimize data types: Review data type decisions and optimize data types where possible (example: use integer instead of text for an enum column)
1. Partitioning: Apply a partitioning scheme if there is a common access dimension.
1. Normalization: Review relational modeling and apply normalization techniques to remove duplicate data
1. Vertical table splits: Review column usage and split table vertically.
1. Externalize: Move large data types out of the database entirely. For example, JSON documents, especially when not used for filtering, may be better stored outside the database,

for example, in object storage.

NOTE:

While we're targeting to limit physical table sizes, we consider retaining or improving performance a goal, too.

For solutions like normalization, this is a trade-off: Denormalized models can speed up queries when used appropriately, at the expense of table size. When normalizing models, splitting tables or externalizing data, we aim to understand the impact on performance and strive to find a solution to reduce table sizes that doesn't impact performance significantly.

Example efforts

A few examples can be found below, many more are organized under the epic Database efficiency.

- 1. Reduce number of indexes on cibuilds
- 1. Normalize and de-duplicate committer and author details in mergerequestdiffcommits
- 1. Retention strategy for cibuildtracesections
- 1. Implement worker that hard-deletes old CI jobs metadata
- 1. mergerequestdifffiles violates < 100 GB target (epic)

Execution

The epic for ~group::database drives decision making to establish and communicate the target and to identify and propose necessary changes to reach it. Those changes should primarily be driven by the respective stage group owning the data (and the feature using it), with ~group::database to support.

Who

<!-- vale gitlab.Spelling = NO -->

Identifying solutions for offending tables is driven by the GitLab Database Team and respective stage groups.

| Role | Who |
|--------------------|----------------|
| ----- | ----- |
| Author | Andreas Brandl |
| Engineering Leader | Alex Ives |
| Principal Engineer | Thong Kuah |
| Senior Engineer | Maxime Orefice |

Decision log

- ADR-001: Table classification
- ADR-002: Limiting new columns for tables larger than 100 GB
- ADR-003: Limiting new indexes for tables larger than 50 GB

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-
documents/database_size_limits/decisions/001_table_classification.md

Context

Database table sizes significantly impact feature development and maintenance. Currently:

- Developers may not be aware of table sizes when developing features
- Large tables (>100 GB) require additional maintenance effort
- Teams need visibility into tables approaching size limits

Decision

We classify tables into size categories to help track and manage growth:

| Classification | Size Range |
|----------------|---------------------|
| ----- | ----- |
| Small | Less than 10 GB |
| Medium | 10 GB to 50 GB |
| Large | 50 GB to 100 GB |
| Over limit | Greater than 100 GB |

These classifications are stored in the database table data dictionary as tablesize.

Consequences

- We create a monthly chore to keep the tablesize in the data dictionary up to date, which can be automated.
- Size classifications help inform development decisions

SECTION: engineering/architecture/design-
documents/database_size_limits/decisions/002_limit_new_columns.md

Context

GitLab aims to maintain table sizes under 100 GB for optimal performance and manageability. Table size growth occurs through:

- Adding new columns
- Increasing row counts
- Creating new indexes

While this decision focuses on preventing new column additions, other size control methods like partitioning and data retention policies are handled separately.

Decision

We prevent new column additions to tables larger than 100 GB through:

1. RuboCop enforcement (!173248)
1. Documentation of allowed exceptions (!169164)

Exceptions to this rule are documented in our development guidelines and require explicit approval.

Consequences

- New features may require additional engineering effort.
 - Some features may require architectural changes.
 - Large tables may require partitioning strategies.
 - Data retention policies become more critical.
 - Developers may need to create separate tables for related data (for example, namespacesettings for namespaces).
- This may add some complexity to the database schema.

SECTION: engineering/architecture/design-
documents/database_size_limits/decisions/003_limit_new_indexes.md

Context

Database indexes significantly impact table sizes while playing a crucial role in query performance. As tables grow:

- Indexes are often added to maintain performance
- Each index increases the total table size
- Indexes introduce additional maintenance overhead, including:
 - Write amplification
 - Increased backup size
 - Higher vacuum overhead

Decision

We prevent new indexes to tables larger than 50 GB through:

Exceptions to this rule are documented in our development guidelines and require explicit approval.

These restrictions affect:

- New standalone indexes
- Indexes required for foreign keys
- Composite indexes
- Partial indexes

Consequences

- New foreign key columns are restricted due to required index creation
- Limited ability to optimize queries through indexing
- Reduced options for filtering and searching
- Potential performance degradation for complex queries
- Teams must consider alternative optimization strategies
- Some feature implementations may require redesign

Mitigation strategies

- Consider table partitioning for tables approaching 50 GB
- Implement appropriate data retention policies
- Explore alternative query optimization techniques
- Use existing indexes where possible

SECTION: engineering/architecture/design-documents/database_testing/_index.md

{{< engineering/design-document-header >}}

Notice: This blueprint has been partially implemented. We still plan to iterate on the tooling. The content below is a historical version of the blueprint, written prior to incorporating database testing into our development workflow.

We have identified common themes of reverted migrations and discovered failed migrations breaking in both production and staging even when successfully tested in a developer environment. We have also experienced production incidents even with successful testing in staging. These failures are quite expensive: they can have a significant effect on availability, block deployments, and generate incident escalations. These escalations must be triaged and either reverted or fixed forward. Often, this can take place without the original author's involvement due to time zones and/or the criticality of the escalation. With our increased deployment speeds and stricter uptime requirements, the need for improving database testing is critical, particularly earlier in the development process (shift left).

From a developer's perspective, it is hard, if not unfeasible, to validate a migration on a large enough dataset before it goes into production.

Our primary goal is to provide developers with immediate feedback for new migrations and other database-related changes tested on a full copy of the production database, and to do so with high levels of efficiency (particularly in terms of infrastructure costs) and security.

Current day

Developers are expected to test database migrations prior to deploying to any environment, but we lack the ability to perform testing against large environments such as GitLab.com. The developer database migration style guide provides guidelines on migrations, and we focus on validating migrations during code review and testing in CI and staging.

The code review phase involves Database Reviewers and Maintainers to manually check the migrations committed. This often involves knowing and spotting problematic patterns and their particular behavior on GitLab.com from experience. There is no large-scale environment available that allows us to test database migrations before they are being merged.

Testing in CI is done on a very small database. We mainly check forward/backward migration consistency, evaluate RuboCop rules to detect well-known problematic behaviors (static code checking) and have a few other, rather technical checks in place (adding the right files etc). That is, we typically find code or other rather simple errors, but cannot surface any data related errors - which are also typically not covered by unit tests either.

Once merged, migrations are being deployed to the staging environment. Its database size is less than 5% of the production database size as of January 2021 and its recent data distribution does not resemble the production site. Oftentimes, we see migrations succeed in staging but then fail in production due to query timeouts or other unexpected problems. Even if we caught problems in staging, this is still expensive to reconcile and ideally we want to catch those problems as early as possible in the development cycle.

Today, we have gained experience with working on a thin-cloned production database (more on this below) and already use it to provide developers with access to production query plans, automated query feedback and suggestions with optimizations. This is built around Database Lab and Joe, both available through Slack (using ChatOps) and postgres.ai.

Vision

As a developer:

1. I am working on a GitLab code change that includes a data migration and changes a heavy database query.
1. I push my code, create a merge request, and provide an example query in the description.
1. The pipeline executes the data migration and examines the query in a large-scale environment (a copy of GitLab.com).
1. Once the pipeline finishes, the merge request gets detailed feedback and information about the migration and the query I provided. This is based on a full clone of the production database with a state that is very close to production (minutes).

For database migrations, the information gathered from execution on the clone includes:

- Overall runtime.
- Detailed statistics for queries being executed in the migration (normalizing queries and showing their frequencies and execution times as plots).
- Dangerous locks held during the migration (which would cause blocking situations in production).

For database queries, we can automatically gather:

- Query plans along with visualization.
- Execution times and predictions for production.
- Suggestions on optimizations from Joe.
- Memory and IO statistics.

After having gotten that feedback:

1. I can go back and investigate a performance problem with the data migration.
1. Once I have a fix pushed, I can repeat the above cycle and eventually send my merge request for database review. During the database review, the database reviewer and maintainer have all the additional generated information available to them to make an informed decision on the performance of the introduced changes.

This information gathering is done in a protected and safe environment, making sure that there is no unauthorized access to production data and we can safely execute code in this environment.

The intended benefits include:

- Shifting left: Allow developers to understand large-scale database performance and what to expect to happen on GitLab.com in a self-service manner
- Identify errors that are only generated when working against a production scale dataset with real data (with inconsistencies or unexpected patterns)
- Automate the information gathering phase to make it easier for everybody involved in code review (developer, reviewer, maintainer) by providing relevant details automatically and upfront.

Technology and next steps

We already use Database Lab from postgres.ai, which is a thin-cloning technology. We maintain a PostgreSQL replica which is up to date with production data but does not serve any production traffic. This runs Database Lab which allows us to quickly create a full clone of the production dataset (in the order of seconds).

Internally, this is based on ZFS and implements a "thin-cloning technology". That is, ZFS snapshots are being used to clone the data and it exposes a full read/write PostgreSQL cluster based on the cloned data. This is called a thin clone. It is rather short lived and is going to be destroyed again shortly after we are finished using it.

A thin clone is fully read/write. This allows us to execute migrations on top of it.

Database Lab provides an API we can interact with to manage thin clones. In order to automate the migration and query testing, we add steps to the gitlab/gitlab-org CI pipeline. This triggers automation that performs the following steps for a given merge request:

1. Create a thin-clone with production data for this testing session.
1. Pull GitLab code from the merge request.
1. Execute migrations and gather all necessary information from it.
1. Execute query testing and gather all necessary information from it.
1. Post back the results of the migration and query testing to the merge request.
1. Destroy the thin-clone.

Short-term

The short-term focus is on testing regular migrations (typically schema changes) and using the existing Database Lab instance from postgres.ai for it.

In order to secure this process and meet compliance goals, the runner environment is treated as a production environment and similarly locked down, monitored and audited. Only Database Maintainers have access to the CI pipeline and its job output. Everyone else can only see the results and statistics posted back on the merge request.

We implement a secured CI pipeline on Internal GitLab for Operations that adds the execution steps outlined above. The goal is to secure this pipeline to solve the following problem:

Make sure we strongly protect production data, even though we allow everyone (GitLab team/developers) to execute arbitrary code on the thin-clone which contains production data.

This is in principle achieved by locking down the GitLab Runner instance executing the code and its containers on a network level, such that no data can escape over the network. We make sure no communication can happen to the outside world from within the container executing the GitLab Rails code (and its database migrations).

Furthermore, we limit the ability to view the results of the jobs (including the output printed from code) to Maintainer and Owner level on the Internal GitLab for Operations pipeline and provide only a high level summary back to the original MR. If there are issues or errors in one of the jobs run, the database Maintainer assigned to review the MR can check the original job for more details.

With this step implemented, we already have the ability to execute database migrations on the thin-cloned GitLab.com database automatically from GitLab CI and provide feedback back to the merge request and the developer. The content of that feedback is expected to evolve over time and we can continuously add to

this.

We already have an MVC-style implementation for the pipeline for reference and an example merge request with feedback from the pipeline.

The short-term goal is detailed in this epic.

Mid-term - Improved feedback, query testing and background migration testing

Mid-term, we plan to expand the level of detail the testing pipeline reports back to the merge request and expand its scope to cover query testing, too. By doing so, we use our experience from database code reviews and using thin-clone technology and bring this back closer to the GitLab workflow. Instead of reaching out to different tools (postgres.ai, joe, Slack, plan visualizations, and so on) we bring this back to GitLab and working directly on the merge request.

Secondly, we plan to cover background migrations testing, too. These are typically data migrations that are scheduled to run over a long period of time. The success of both the scheduling phase and the job execution phase typically depends a lot on data distribution - which only surfaces when running these migrations on actual production data. In order to become confident about a background migration, we plan to provide the following feedback:

1. Scheduling phase - query statistics (for example a histogram of query execution times), job statistics (how many jobs, overall duration, and so on), batch sizes.
1. Execution phase - using a few instances of a job as examples, we execute those to gather query and runtime statistics.

Long-term - incorporate into GitLab product

There are opportunities to discuss for extracting features from this into GitLab itself. For example, annotating the merge request with query examples and attaching feedback gathered from the testing run can become a first-class citizen instead of using merge request description and comments for it. We plan to evaluate those ideas as we see those being used in earlier phases and bring our experience back into the product.

An alternative discussed: Anonymization

At the core of this problem lies the concern about executing (potentially arbitrary) code on a production dataset and making sure the production data is well protected. The approach discussed above solves this by strongly limiting access to the output of said code.

An alternative approach we have discussed and abandoned is to "scrub" and anonymize production data. The idea is to remove any sensitive data from the

database and use the resulting dataset for database testing. This has a lot of downsides which led us to abandon the idea:

- Anonymization is complex by nature - it is a hard problem to call a "scrubbed clone" actually safe to work with in public. Different data types may require different anonymization techniques (for example, anonymizing sensitive information inside a JSON field) and only focusing on one attribute at a time does not guarantee that a dataset is fully anonymized (for example join attacks or using timestamps in conjunction to public profiles/projects to de-anonymize users by there activity).
- Anonymization requires an additional process to keep track and update the set of attributes considered as sensitive, ongoing maintenance and security reviews every time the database schema changes.
- Annotating data as "sensitive" is error prone, with the wrong anonymization approach used for a data type or one sensitive attribute accidentally not marked as such possibly leading to a data breach.
- Scrubbing not only removes sensitive data, but it also changes data distribution, which greatly affects performance of migrations and queries.
- Scrubbing heavily changes the database contents, potentially updating a lot of data, which leads to different data storage details (think MVC bloat), affecting performance of migrations and queries.

SECTION: engineering/architecture/design-documents/database_traffic_replay/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

Develop comprehensive tooling to capture, store, and replay SQL query traffic from GitLab.com. This solution will implement a lightweight query forwarding mechanism within GitLab that sends SQL queries to an external service with minimal performance impact on Rails and Sidekiq processes. Combined with purpose-built replay utilities, this system will enable performance testing, capacity planning, and database architecture evaluation, allowing us to simulate production loads at variable speeds, identify saturation and contention points, assess potential database configuration changes, and validate sharding strategies without adversely affecting production systems.

Motivation

This tool will allow us to collect and measure our database capacity. This will effectively settle questions about the capacity of both our current setup, as well as the effectiveness of other mitigations such as changes to anything in our production database infrastructure.

Goals

1. Negligible performance impact on production Rails/Sidekiq processes.
2. Increased confidence in database scaling decisions and configuration changes.
3. Enhanced ability to identify and mitigate performance bottlenecks proactively.
4. Improved database architecture testing capabilities without production risk.

Non-Goals

1. This is not a backup tool.
2. Not for data analysis, doesn't run continuously, not load bearing for any other uses.
3. We do not expect to converge on exactly the same database state that occurred in production. Specifically we are concerned with database performance under load, not correctness or data consistency.
4. We expect captures to be not entirely consistent. Some queries will fail to execute during replay.

Proposal

We will capture all query traffic from the gitlab application, and replay it against the benchmarking environment. We will do this for a period of time, generally not to exceed an hour, and will use it to verify configuration changes to database hosts.

We will also support shrinking the traffic replay into a smaller period of time to simulate higher production load that the application might see in the future.

Design and implementation details

We will capture query data from rails nodes, publish it to a pubsub topic, then aggregate the query data and persist it to a bucket.

mermaid

flowchart TD

```
subgraph DataPlane1["Data Plane"]
  subgraph GitlabProd["gitlab-production"]
    GitlabRails1["Gitlab Rails"]
    GitlabRails2["Gitlab Rails"]
    GitlabRails3["Gitlab Rails"]
  end
end

subgraph PubSub["PubSub"]
  Topic["Topic"]
  PS["Pull Subscription"]
  note1["Each message in the pubsub topic and subscriber is a query + metadata."]
end

subgraph Dataflow["DataFlow"]
```

```

    Subscriber["Subscriber"]
    note2["Inputs messages, outputs
    formatted querydata into the bucket."]
end

subgraph Bucket["Bucket"]
    CurrentFolder["Folder for this run"]
    PreviousFolder["Previous Run"]
    note3["Data is stored with same permission
    and retention restrictions as WAL data"]
end
end

%% Define connections
GitlabRails1 -->|"Queries"| Topic
GitlabRails2 -->|"Queries"| Topic
GitlabRails3 -->|"Queries"| Topic
Topic --> PS
PS --> Subscriber
Subscriber --> CurrentFolder

%% Styling
classDef sectionHeader fill:f9f,stroke:333,stroke-width:2px
classDef pauseStep fill:ffe6cc,stroke:d79b00,stroke-width:1px
classDef pipelineStep fill:d5e8d4,stroke:82b366,stroke-width:1px

class QueryCapture,QueryReplay sectionHeader
class PauseA,PauseB pauseStep
class
PipelineStageA,PipelineStageB,PipelineStageC,MachineCreation,ReplayPerformed,MachinesDeleted
pipelineStep

```

To replay, we will load the query data from the bucket, and replay it against a benchmarking database restored from production at the time of the capture, using the same number of connections as were originally used in production.

```

mermaid
flowchart TD
    subgraph DataPlane2["Data Plane"]
        subgraph Benchmark["gitlab-db-benchmarking"]
            MachineCreation["Machines Created"]
            GPRD["GPRD Snapshot used for machine creation is calculated on timestamp of folder of query data being replayed."]
            WAL["WAL is replayed up until start LSN of the query data capture, then stopped."]
        end
    end

    subgraph PipelineStageA["Trigger Pipeline Stage A"]
        StageAVars["input variables include:"]
        -["foldername of querydata we want to replay"]
    end

```

- how many replay nodes we want to create
- primary database instance only or also replicas
- if we want database replicas, how many?
- if we want to override default machine sizes, machine types etc"]

end

PauseA["Pause until next step manually triggered
A good time for engineers to make any other manual changes to environment"]

subgraph PipelineStageB["Trigger Pipeline Stage B"]
 StageBVars["input variables include:
 - replay speed (1x, 1.2x, 2x, etc)
 - write only or read & write

Engineers can choose not to run Stage B if they just wanted
 an environment created and do not need a querydata replay"]

end

subgraph ReplayPerformed["Replay Performed"]
 ReplayProcess["Query Data will be gathered from GPRD bucket then replayed"]

end

PauseB["Pause until next step manually triggered
A good time for engineers to make any other checks on the environment"]

subgraph PipelineStageC["Trigger Pipeline Stage C"]
 StageCNote["- Pipeline C needs to be able to be run even if Pipelines A and B
 either a) were not run or b) failed"]

end

subgraph MachinesDeleted["Machines Deleted"]
 DeleteNote["Query Data will NOT be deleted from GPRD bucket"]

end

end

PipelineStageA -->["Creates"| MachineCreation
 MachineCreation --> PauseA
 PauseA --> PipelineStageB
 PipelineStageB -->["Executes"| ReplayPerformed
 ReplayPerformed --> PauseB
 PauseB --> PipelineStageC
 PipelineStageC -->["Cleans up"| MachinesDeleted

%% Styling
 classDef sectionHeader fill:f9f,stroke:333,stroke-width:2px
 classDef pauseStep fill:ffe6cc,stroke:d79b00,stroke-width:1px
 classDef pipelineStep fill:d5e8d4,stroke:82b366,stroke-width:1px

class QueryCapture,QueryReplay sectionHeader
 class PauseA,PauseB pauseStep

```
class
PipelineStageA,PipelineStageB,PipelineStageC,MachineCreation,ReplayPerformed,MachinesDeleted
pipelineStep
```

Security and Retention

Note that the recorded traffic capture would contain RED data.

We plan to treat this data the same way that we treat the WAL archives we already store for backup / restore validation, which contain roughly the same data in a different format (the results of queries rather than their text). Specifically we'll store the query text in a traffic-capture bucket in the production environment, similar to WAL data, and perform replay testing in the db-benchmarking environment, also similar to how we currently run other testing. We will retain capture data for only 14 days, before it will be deleted by a lifecycle rule on the traffic-capture bucket

The traffic-capture bucket will inherit most of its permissions from gitlab-production gcp project rules or gitlab.com gcp organization rules. However it will also be accessible by a traffic-capture service account which will have the roles/storage.objectUser permission necessary to perform traffic capture, and a traffic-replay service account which will have the roles/storage.objectUser permission necessary to perform traffic replay.

We are using GCP's native PubSub, so message encryption is handled by default, and service accounts are used for authentication.

A service account representing Gitlabs rails nodes will receive the role roles/pubsub.publisher on the traffic capture pub/sub topic to allow it to add messages to the pubsub queue.

The traffic-capture service account will receive the roles

- roles/pubsub.subscriber on the traffic-capture pubsub subscription and the traffic-capture pubsub topic
- roles/pubsub.viewer on the traffic-capture pubsub topic
- roles/dataflow.worker for the entire project. Unfortunately this necessary role can only be assigned on an entire project, -Provides%20the%20permissions). Luckily, traffic-capture is the only functionality which uses Dataflow in gitlab-production.

Alternative Solutions

1. We could capture query traffic at the connection pooler level, using something like pgcat or pgdog.
 - We aren't yet running such a pooler in production (we're currently running pgbouncer) and this tool will be useful to evaluate a change in connection pooler.
2. We could use a tool already built, such as <https://github.com/gocardless/pgreplay-go>.
 - pgreplay-go and similar tools capture data from the postgres log file, but that won't work at our scale - the volume of query text would exceed the capacity of a disk very quickly.

SECTION: engineering/architecture/design-documents/data_insights_platform/_index.md

{{< engineering/design-document-header >}}

Summary

Data Insights Platform is a unified abstraction to ingest, process, persist & query analytical data streams generated across GitLab enabling our ability to compute business insights across the product.

The motivation behind building a centralized data platform manifests from our work within the Product Usage Data Unification Working Group wherein we established a need for consolidating all our current multiple ways of collecting analytical & product usage data generated across GitLab into a single cohesive abstraction with a few key goals:

- Stitch together multiple pieces of analytics-specific functionality into one cohesive product feature backed by a single abstraction/system.
- Build the ability to ingest all analytical data into a centralized, scalable unit of processing & querying such data without compromising ease of use for stakeholders.

Motivation

- Build within the product
 - A core design tenet with the Platform is to build it within the product - with zero external dependencies and deep-integration with GitLab/GDK similar to existing services such as Gitaly.
 - Ensure the availability of the Platform across all environments we run a GitLab instance in .com, Dedicated and Self-managed. The design of the Platform should also allow deploying cluster-instances for Cells as applicable.
- Streamlined user experience
 - Abstract away undue complexity from a Platform user or developer. The Platform should perform its advertised features without the end-user having to reinvent the wheel every time they need to gather analytical data and/or be able to query/process it once collected.
- Feature consolidation
 - Consolidate any analytics-related events-emitting system/feature under the same logical abstraction minimising the work developers need to do to add new functionality.
- Production-ready
 - Ensure the Platform is secure, scalable, reliable & maintainable for the foreseeable future.
- Future-proof implementation
 - Ensure the Platform is sufficiently modularised to allow changing its dependencies.
 - Ensure all ingested data can be routed to multiple systems across our technology stack depending on the needs of the product.

Goals

- Consolidate the various event-collection mechanisms and simplify supporting infrastructure and/or systems - inline with our decisions in the following Architecture Design Records (ADRs) from the working group:
 - Do not use proprietary components for collecting and processing event data.
 - Replace internal usage of Snowplow with dogfooding of Data Insights Platform.
 - Build a single events routing interface.
 - Build GitLab Data Connector integration within a single codebase.
 - The existence of a centralized data platform helps enable the following key outcomes with the full data lifecycle in mind:
 - Unified data collection and storage.
 - Efficient data movement across systems.
 - Secure data access and governance.
 - Data visualization and exploration.
- > Our latest update on Data Unification and Insights presents these in further details.
- Broadly speaking, the Platform should enable a paved path for us to build an event-driven platform within the product.

Proposal

Data Insights Platform is a logical abstraction comprising a set of components functioning together to structurally form a scalable data-pipeline that allow us to ingest, process and persist large volumes of analytical data into designated datastores from where it can be queried efficiently.

Following is a brief description of each underlying component. Note, we have also compiled architecture blueprints for some of these individually to ensure we can deep-dive into further details of each component and gather valuable feedback from a wider audience.

- Ingestor(s): Single ingestion mechanism for supported event types - which can be run both locally for development & as a cluster when in production. This layer is intentionally stateless to allow horizontal scalability to allow ingesting large data volumes. We aim to build the support for ingesting the following event-types:
 - Snowplow (current iteration)
 - CloudEvents (next iteration)
 - Service Ping (future iteration)
- Siphon: Custom, in-house Change Data Capture (CDC) implementation to logically replicate data from Postgres into other systems such as ClickHouse.
 - Architecture Blueprint introducing Siphon.
- NATS: Embedded/Distributed system to buffer incoming data prior to processing/enriching.
 - Architecture Blueprint introducing NATS as a messaging layer within GitLab
- Enrichment Framework: Custom framework to enrich incoming data with the ability to communicate with external components such as GitLab API or Data Catalog for metadata.

- Supported enrichments include operations such as pseudonymization or redaction of sensitive parts of ingested data, PII detection, parsing client useragent strings, etc.
- Exporters: Custom implementations that help ship ingested data into designated persistent stores for further querying/processing.
 - ClickHouse Exporter: ClickHouse is our designated persistent database which helps us persist all analytical data ingested by the Platform and query from using the Query API.
 - S3/GCS Exporter: Having data shipped to S3/GCS helps land data into Snowflake powering our current analytical query-workflows using Snowflake & Tableau.
- ClickHouse: External persistent database that allows for durable persistence and advanced OLAP querying capabilities for all analytical data ingested within the Platform.
- Query API: Custom, semantic querying layer for clients/UIs to query data persisted within ClickHouse.
 - Architecture Blueprint introducing Platform's querying capabilities.

Identified use-cases

Following is a detailed set of use-cases that benefit from the existence of a centralized data platform within the product.

| Teams/areas | Use-cases | Expected scale |
|--------------------------------|---|---|
| Enterprise Data/Infrastructure | Logically replicating data out of Postgres. | ~100GB of new data ingested from Postgres per hour. We'll also need to scale with Cells architecture. |
| Platform Insights | Ingesting & processing large amounts of analytics data in real-time, then persisting it into ClickHouse. | Product Usage Data: 1200 events/sec (100M daily), ~15GB/hour, expected to increase with increase in instrumentation + 2.5x when pursuing event-level data collection from the customer's domain (about 300M events daily). |
| Machine Learning | Extracting & processing events/features from GitLab data to create training/test datasets for ML models at scale. | |
| Security & Compliance | Extracting and actioning upon events, e.g., audit events, from GitLab data in near real-time. | Audit events saved to database per day (excludes streaming only audit events) - ~0.6M records are created per day (audit event coverage has stalled due to load on Postgres, this is likely to increase if this migrates over to an event pipeline) |
| | | Streaming only audit events generated per day (this excludes audit events saved to DB) - ~35M streaming only events are created per day. |
| | | Total estimates would be at ~40M events per day. See Kibana dashboard (internal). |
| Product | Generic Events Platform to asynchronously process data, events & tasks. Ingesting & processing external data via webhooks as a service. | |
| Product | Implementing real-time analytics features on top of ClickHouse. | Data volume: Similar or less than what we observe with the PostgreSQL databases. Depends on how many tables we replicate (Siphon) to ClickHouse. Enqueued event count: significantly lower as we're batching the CDC events into packages. |
| Plan | JIRA Compete Strategy | |
| | Optimize | Ingesting JIRA data |
| Fulfillment | Consumption Billing for Dedicated Hosted Runners | |

Deployments

With the intention to make the Data Insights Platform available to every GitLab installation, we aim to build necessary support for the following deployment models:

| Deployment type | Proposed topology |
|-----------------|--|
| GDK | Running local to the installation |
| .com | One or more dedicated cluster(s) |
| Cells | One or more clusters subject to cells-topology |
| Dedicated | One dedicated cluster per instance |
| Self-Managed | Standalone cluster subject to distribution/instance-sizing |

Rollout roadmap

| Deliverables | Timeline | Description |
|---------------------------------|----------|-------------------------------------|
| Pre-staging/Testing | FY26Q1 | Deployed to GKE using this project. |
| .com Staging | TBD | - |
| .com Production | TBD | - |
| Gradual rollout to Dedicated | TBD | - |
| Gradual rollout to Self-managed | TBD | - |

Opportunities looking forward

- Sharing analytical data externally, including with our customers wherein any ingested or enriched data and insights derived from it can be shared with all stakeholders as permissible within our SAFE framework.
- Ability to generate insights & metrics around business continuity faster.
- Building flexible/loose coupling between analytical parts of the product, allowing components to be switched should our future business needs change.
- Scalable data ingestion pipeline for augmenting our AI initiatives across GitLab, for example, help power RAG ingestion pipelines.
- Gathering operational insights from running software systems at GitLab, existing proposal at Proposal: GitLab Observability Component - Structured Events.

Design & Implementation

Monolithic implementation

- The Platform must be developed within a single project using Go.
- The Platform must be developed and packaged as a single binary allowing it to run standalone and be well-integrated with GDK for local development.
- The Platform must also support running in a distributed fashion - scaling out as multi-service deployments when needed. The implementation should allow for it to be broken down into multiple services by using flags wherein each service can run independently on their own compute pools for independent scalability.

NATS + JetStream as a centralised data buffer

- Considering NATS can both be embedded within an application and scale out as a cluster when needed, NATS makes it trivial for us to deploy NATS clusters across all our reference

architectures we decide to run the Platform in.

- With NATS forming the data backbone of the Platform, it makes it trivial for us to:
 - Be able to ingest data from multiple event-sources,
 - Be able to centrally & dynamically process & enrich it without having to send data across systems,
 - Be able to route it to other data-related systems as needed.

Minimal external dependencies

- With the only direct storage dependency being ClickHouse, it should be possible to develop & test all analytical features within the Platform completely locally within GDK.

Support for common standards

- Snowplow for structured/tracked events via a consolidated event instrumentation layer.
- CloudEvents for custom structured events.

Looking forward, we should also be able to assimilate other currently used formats such as Service Ping or remodel them with one of aforementioned supported common standards.

Architecture

As suggested in the diagram above, the Platform is essentially a composition of multiple components functioning together to resemble a scalable data pipeline capable of ingesting, processing and persisting data durably in one or more persistent datastores. From the diagram, data ingestion starts at the ingesters which pushes it into one or more NATS streams - subject to configured partitioning and/or data retention policies. In a following step, the enricher framework dequeues data as necessary, performs all configured enrichments and writes it back to one or more designated NATS streams. Following this, configured exporters pick up enriched data from NATS and ship it to one or more downstream datastores. This is where any necessary fan-out of data happens - moving all enriched data from a single source into systems where that data might be needed. All data querying can now happen from these datastores and/or systems.

Having NATS available in-between the producers and consumers of these data streams helps decouple the two - allowing them to scale independently of each other, considering data volumes at ingress can be much higher than what we can process and/or persist durably. This architecture also accounts for handling any backpressure within the pipeline without affecting the health of involved components. We intend for all data ingestion to be highly performant while downstream processing or exporting of data happens asynchronously in a more regulated fashion subject to downstream systems and/or their operational capacity.

Security

The following sections describe how we ensure the Platform and the data ingested or queried from it remains secured.

Authentication & Authorization

- All internal communication in between GitLab (monolith) and the Platform must be encrypted using TLS.
- For Platform's data ingestion and querying endpoints, we intend to defer most authentication & authorization decisions onto GitLab (monolith) itself. When ingesting data, all events will be forwarded to the endpoint by using the event forwarder implementation within the Rails application. When querying data, all requests will also be pre-authorized by the monolith and the corresponding service(s) within GitLab will use a shared-token to talk to the Platform (similar to Gitaly).
- Platform's internal interactions with NATS will be authenticated and authorized as necessary.
- Platform's internal interaction with ClickHouse will be authenticated and authorized as necessary.

Encryption

- Data while at rest within NATS will remain encrypted by using filesystem encryption.
- Data persisted durably in ClickHouse will also remain encrypted. For ClickHouse Cloud, we can start with Transparent Data Encryption (TDE) or Customer Managed Encryption Keys (CMEK) as applicable.

Auditing & Logging

- For the environments we manage, logs from all Platform components must be shipped to centralised logging as applicable.

Operations

- The single-binary implementation allows scaling out each component of the Platform individually. When needed, each of ingester, enricher & exporter can be scaled and run on a segregated node-pool.
- Internally, we aim to shard incoming data across N distinct NATS streams which can then be consumed by multiple instances of the enricher/exporter processes.

Scalability

- On our testing instance, we ran the following configuration:

text

```
--replicas=5",
--streams=5",
--enricher-workers=10",
--flush-batch-size=1000",
--flush-interval-seconds=10",
```

for the single-binary running a 5-replica statefulset (2 vCPU, 2GB memory each) on a GKE cluster with each node of the type c2d-standard-16 and were able to ingest, enrich and persist Snowplow events within ClickHouse at about 6000 events per second on ingress and ~1600 events per second landed on ClickHouse.

- Drawing from our experience with running the current Snowplow infrastructure for GitLab, we

intend our first iteration of the Platform to scale & handle upto 500 million events per day with upto 1 million events per minute at peak. From an infrastructure footprint standpoint, we intend for our current traffic volumes to be handled with reasonably lesser resources than we employ right now.

- Siphon has also been confirmed to be replicating logical replication events at ~5700 events per second with Postgres being our current bottleneck. More details of Siphon's performance testing is available in this issue. We also tested failing over the underlying NATS cluster during these tests and the complete dataset recovered successfully as soon as NATS service was back functional.

- A single cluster is the top-level deployment unit of the Platform, as described in the architecture diagram above. If needed, we can deploy one or more clusters of the Platform to accommodate for scale and/or tenant-isolation per cluster. For example, when ingesting data from Self-managed or Dedicated GitLab instances, it can be routed to different Platform clusters to ensure one's traffic volumes does not interfere with the resources of another.

Monitoring

- For components developed internally, we must add sufficient instrumentation and expose telemetry as Prometheus metrics.
- For the environments we manage, metrics from all Platform components must be shipped to our internal monitoring backends (Mimir) as applicable and added to our internal dashboards.

SECTION: engineering/architecture/design-
documents/data_insights_platform_hierarchical_data_retrieval_optimization/_index.md

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

With the Data Insights Platform (DIP), we can optimize existing GitLab features and improve scalability by leveraging alternative database systems for data querying. The platform will also provide real-time analytical capabilities through ClickHouse, which is well-suited for processing large volumes of data.

Proposal

Leverage the Data Insights Platform, utilizing Siphon for efficient data synchronization with ClickHouse. The ingested data in ClickHouse will be enhanced through the use of materialized views, specifically designed to support hierarchical queries over large datasets.

Problem statement

Querying data in the group hierarchy involves the following steps when the PostgreSQL database is used:

1. Locate the group and its subgroups (unbounded, there is no application limit on the number of allowed subgroups).
2. For each item, look up the associated resource (for example issues).
3. Apply additional filters and joins.
4. Count or sort the records.
5. Present the results to the user.

A simplified database query:

```
sql
SELECT
    issues.
FROM issues
WHERE
    -- this subquery may contain several thousands of projects
    projectid IN (SELECT id from projects WHERE ...)
ORDER BY createdat, id
LIMIT 21
```

An example database query from the codebase:

```
sql
SELECT
    "issues".
FROM
    "issues"
    INNER JOIN "projects" ON "projects"."id" = "issues"."projectid"
    LEFT JOIN projectfeatures ON projects.id = projectfeatures.projectid
WHERE (NOT EXISTS (
    SELECT
        1
    FROM
        "bannedusers"
    WHERE (issues.authorid = bannedusers.userid)))
AND "projects"."namespaceid" IN (SELECT namespaces.id FROM (
    SELECT
        namespaces.traversalids[arraylength(namespaces.traversalids, 1)] AS id FROM "namespaces"
    WHERE
        "namespaces"."type" = 'Group'
        AND (traversalids @> ('{9970}')) namespaces)
    AND (EXISTS (
        SELECT
            1
        FROM
            "projectauthorizations"
        WHERE
```

```
"projectauthorizations"."userid" = 4156052
AND (projectauthorizations.projectid = projects.id)
AND (projectauthorizations.accesslevel >= 10))
OR projects.visibilitylevel IN (10, 20))
AND ("projectfeatures"."issuesaccesslevel" IS NULL
OR "projectfeatures"."issuesaccesslevel" IN (20, 30)
OR ("projectfeatures"."issuesaccesslevel" = 10
AND EXISTS (
SELECT
```